Collections et tuples en Swift

420-249AH - Programmation mobile iOS

1. Pourquoi utiliser des collections ?

Dans une application mobile, on travaille rarement avec une seule valeur. On doit souvent gérer plusieurs éléments du même type.

Exemples :

- Liste de messages
- Liste de contacts
- Liste de notifications
- Historique de positions GPS
- Produits dans un panier
- Applications favorites
- Valeurs de capteurs
- Paramètres de configuration

Plutôt que de créer :

```swift
let message1 = "Bonjour"
let message2 = "Bienvenue"
let message3 = "Nouvelle notification"
```

on peut regrouper les valeurs dans une collection.

2. Les principales collections Swift

Swift propose principalement :

```swift
Array
Dictionary
Set
```

Chaque collection répond à un besoin différent.

Collection	Rôle
`Array`	Collection ordonnée
`Dictionary`	Association clé / valeur
`Set`	Collection de valeurs uniques

Nous verrons également les tuples, qui permettent de regrouper plusieurs valeurs, même de types différents.

3. Array — les tableaux

Un Array permet de conserver plusieurs valeurs dans un ordre précis.

Exemple :

```
var notifications: [String] = [
    "Nouveau message",
    "Mise à jour disponible",
    "Batterie faible"
]
```

On peut visualiser :

Index	Valeur
0	Nouveau message
1	Mise à jour disponible
2	Batterie faible

4. Déclarer un tableau

Syntaxe explicite :

```
var noms: [String] = [
    "Alex",
    "Marie",
    "Sam"
]
```

Swift peut également utiliser l'inférence de type :

```
var noms = [
    "Alex",
    "Marie",
    "Sam"
]
```

Swift comprend que le tableau est de type :

```
[String]
```

5. Tableau d'entiers

```
var niveauxBatterie: [Int] = [
    100,
    85,
    70,
```

```
    55
]
```

Type :

```
[Int]
```

6. Tableau de nombres décimaux

```
var temperatures: [Double] = [
    21.5,
    22.1,
    20.8
]
```

Type :

```
[Double]
```

7. Créer un tableau vide

```
var messages: [String] = []
```

ou :

```
var messages = [String]()
```

Le tableau existe, mais ne contient encore aucun élément :

```
[]
```

8. Ajouter un élément

Pour ajouter une valeur à la fin du tableau :

```
var notifications = [
    "Nouveau message",
    "Batterie faible"
]

notifications.append("Mise à jour disponible")
```

Le tableau devient :

```
Nouveau message
Batterie faible
Mise à jour disponible
```

9. Ajouter plusieurs éléments

On peut utiliser :

```
append(contentsOf:)
```

Exemple :

```
notifications.append(
    contentsOf: [
        "Connexion perdue",
        "Synchronisation terminée"
    ]
)
```

10. Insérer un élément

Pour insérer une valeur à une position précise :

```
notifications.insert(
    "Notification urgente",
    at: 0
)
```

La nouvelle valeur devient le premier élément.

11. Accéder à un élément

Les indices commencent à :

```
0
```

Exemple :

```
let premierMessage = notifications[0]
```

On peut également écrire :

```
print(notifications[1])
```

12. Attention aux indices invalides

Supposons :

```
let messages = [
    "A",
    "B",
    "C"
]
```

Les indices valides sont :

```
0
1
2
```

Ceci est incorrect :

```
print(messages[3])
```

Attention : l'application peut provoquer une erreur d'exécution. Il faut donc être prudent avec les indices.

13. count

count retourne le nombre d'éléments.

```
let notifications = [
    "Message",
    "Batterie",
    "Mise à jour"
]

print(notifications.count)
```

Résultat :

```
Console
3
```

14. isEmpty

Pour vérifier si un tableau est vide :

```
if notifications.isEmpty {
    print("Aucune notification")
} else {
    print("\(notifications.count) notification(s)")
}
```

15. Premier et dernier élément

Swift propose :

```
first
last
```

Exemple :

```
let premier = notifications.first
```

```swift
let dernier = notifications.last
```

Attention : first et last retournent des optionnels. Pourquoi ? Parce qu'un tableau peut être vide.

16. Utiliser first avec if let

```swift
if let premiereNotification = notifications.first {
    print("Première notification : \(premiereNotification)")
}
```

On réutilise ici directement les optionnels vus précédemment.

17. Modifier un élément

```swift
var modes = [
    "Clair",
    "Sombre",
    "Automatique"
]

modes[0] = "Mode clair"
```

Le tableau devient :

```
Mode clair
Sombre
Automatique
```

18. Supprimer un élément

Supprimer selon l'indice :

```swift
notifications.remove(at: 1)
```

Supprimer le dernier élément :

```swift
notifications.removeLast()
```

Supprimer tous les éléments :

```swift
notifications.removeAll()
```

19. Parcourir un tableau

La boucle for-in est particulièrement adaptée aux collections.

```swift
let notifications = [
    "Message reçu",
```

```
    "Batterie faible",
    "Mise à jour disponible"
]

for notification in notifications {
    print(notification)
}
```

Résultat :

```
Console
Message reçu
Batterie faible
Mise à jour disponible
```

20. Exemple — Liste d'applications

```
let applications = [
    "Messages",
    "Plans",
    "Photos",
    "Musique"
]

for application in applications {
    print("Application : \(application)")
}
```

Résultat :

```
Console
Application : Messages
Application : Plans
Application : Photos
Application : Musique
```

21. Parcourir avec l'indice

On peut utiliser :

```
enumerated()
```

Exemple :

```
let pages = [
    "Accueil",
    "Profil",
    "Paramètres"
]
```

```
for (index, page) in pages.enumerated() {
    print("\(index) - \(page)")
}
```

Résultat :

```
Console
0 - Accueil
1 - Profil
2 - Paramètres
```

22. Utiliser un index à partir de 1

Pour un affichage plus naturel :

```
for (index, page) in pages.enumerated() {
    print("\(index + 1). \(page)")
}
```

Résultat :

```
Console
1. Accueil
2. Profil
3. Paramètres
```

23. Vérifier si une valeur existe

On peut utiliser :

```
contains()
```

Exemple :

```
let permissions = [
    "camera",
    "location",
    "notifications"
]

if permissions.contains("camera") {
    print("Permission caméra disponible")
}
```

24. Exemple — Suivi du niveau de batterie

```
let batterie = [
    100,
    90,
    75,
    50,
    25,
    10
]

for niveau in batterie {
    if niveau <= 20 {
        print("\(niveau)% : batterie faible")
    } else {
        print("\(niveau)%")
    }
}
```

On combine ici : Array + for + if.

25. Dictionary — dictionnaires

Un dictionnaire stocke des données sous forme :

```
clé → valeur
```

Exemple :

```
var utilisateur: [String: String] = [
    "nom": "Alex",
    "ville": "Montréal",
    "langue": "fr"
]
```

On peut visualiser :

```
nom    → Alex
ville  → Montréal
langue → fr
```

26. Déclarer un dictionnaire

Syntaxe :

```
var profil: [String: String] = [
    "nom": "Alex",
    "courriel": "alex@email.com"
]
```

Le type est :

```
[String: String]
```

Cela signifie : clé → String, valeur → String.

27. Autres types de dictionnaires

```
var niveaux: [String: Int] = [
    "batterie": 85,
    "volume": 60,
    "luminosite": 70
]
```

Type :

```
[String: Int]
```

28. Accéder à une valeur

```
let nom = profil["nom"]
```

Attention : le résultat est un optional. Type :

```
String?
```

Pourquoi ? Parce que la clé demandée peut ne pas exister.

Autre exemple :

```
let ville = profil["ville"]
```

Si la clé existe :

```
Console
Montréal
```

Sinon :

```
Console
nil
```

29. Utiliser if let

```
if let nom = profil["nom"] {
    print("Nom : \(nom)")
} else {
    print("Nom non disponible")
}
```

Encore une fois, les optionnels sont directement utilisés.

30. Valeur par défaut dans un dictionnaire

On peut écrire :

```
let langue = profil["langue"] ?? "fr"
```

Si "langue" n'existe pas, "fr" sera utilisée. Ce réflexe — utiliser ?? pour éviter une valeur absente — sera très fréquent avec les dictionnaires.

31. Ajouter une entrée au dictionnaire

```
var parametres: [String: String] = [
    "theme": "sombre"
]

parametres["langue"] = "fr"
```

Le dictionnaire devient :

```
theme  → sombre
langue → fr
```

32. Modifier une valeur

```
parametres["theme"] = "clair"
```

La valeur associée à theme est remplacée.

33. Supprimer une entrée du dictionnaire

```
parametres["theme"] = nil
```

ou :

```
parametres.removeValue(forKey: "theme")
```

34. Parcourir un dictionnaire

```
let parametres = [
    "theme": "sombre",
    "langue": "fr",
    "notifications": "activees"
]

for (cle, valeur) in parametres {
    print("\(cle) : \(valeur)")
}
```

35. Attention à l'ordre

Contrairement à un tableau, on ne devrait pas utiliser un dictionnaire lorsque l'ordre logique des éléments est essentiel.

Un dictionnaire sert surtout à retrouver une valeur à partir de sa clé.

Exemple :

```
clé "langue"
      ↓
valeur "fr"
```

36. Exemple — Nombre de notifications par application

```swift
var notificationsParApplication: [String: Int] = [
    "Messages": 4,
    "Courriel": 7,
    "Calendrier": 2
]
```

On peut modifier directement une valeur associée à une clé :

```swift
notificationsParApplication["Messages"] = 5
```

Ou l'augmenter à partir de sa valeur actuelle :

```swift
if let nombre = notificationsParApplication["Messages"] {
    notificationsParApplication["Messages"] = nombre + 1
}
```

On combine ici : accès par clé + optionnel + if let + modification.

37. Set — ensembles

Un Set représente une collection de valeurs uniques.

Exemple :

```swift
var permissions: Set<String> = [
    "camera",
    "location",
    "notifications"
]
```

Un Set ne conserve pas de doublons.

38. Pourquoi utiliser un Set ?

Supposons :

```
var permissions: Set<String> = [
    "camera",
    "location",
    "camera"
]
```

Le résultat contient seulement :

```
camera
location
```

La valeur camera n'est stockée qu'une seule fois.

39. Array vs Set

Collection	Comportement
`["camera", "camera", "location"]`	Les doublons sont autorisés (Array)
`Set(["camera", "camera", "location"])`	Résultat logique : camera, location (Set)

40. Créer un Set

```
var appareils: Set<String> = [
    "iPhone",
    "iPad",
    "Apple Watch"
]
```

Créer un ensemble vide :

```
var appareils = Set<String>()
```

41. Ajouter une valeur au Set

```
appareils.insert("Mac")
```

Si "Mac" existe déjà, il ne sera pas ajouté une deuxième fois.

42. Supprimer une valeur du Set

```
appareils.remove("iPad")
```

43. Vérifier l'appartenance d'une valeur

```
if appareils.contains("iPhone") {
    print("iPhone supporté")
}
```

44. Parcourir un Set

```
for appareil in appareils {
    print(appareil)
}
```

Attention : l'ordre d'un Set ne doit pas être considéré comme significatif.

45. Exemple — Permissions accordées

```
var permissionsAccordees: Set<String> = [
    "camera",
    "notifications"
]
```

Vérification :

```
if permissionsAccordees.contains("location") {
    print("Accès à la localisation")
} else {
    print("Permission localisation requise")
}
```

46. Opérations entre ensembles

Les ensembles peuvent être comparés ou combinés.

Exemple :

```
let permissionsRequises: Set<String> = [
    "camera",
    "location"
]

let permissionsAccordees: Set<String> = [
```

```
    "camera",
    "notifications"
]
```

47. Union

L'union regroupe toutes les valeurs.

```
let toutes = permissionsRequises.union(
    permissionsAccordees
)
```

Résultat logique :

```
camera
location
notifications
```

48. Intersection

L'intersection conserve les valeurs présentes dans les deux ensembles.

```
let communes = permissionsRequises.intersection(
    permissionsAccordees
)
```

Résultat :

```
Console
camera
```

49. Différence

```
let manquantes = permissionsRequises.subtracting(
    permissionsAccordees
)
```

Résultat :

```
Console
location
```

Cet exemple est particulièrement intéressant pour vérifier des permissions.

50. Exemple — Permissions manquantes

```
let permissionsRequises: Set<String> = [
    "camera",
    "location",
    "notifications"
]

let permissionsAccordees: Set<String> = [
    "camera",
    "notifications"
]

let permissionsManquantes =
    permissionsRequises.subtracting(
        permissionsAccordees
    )

for permission in permissionsManquantes {
    print("Permission manquante : \(permission)")
}
```

51. Choisir la bonne collection

Utiliser Array

Lorsque : l'ordre est important, les doublons sont possibles, on accède aux éléments par index.

Exemples : Messages, Historique, Liste de produits, Positions GPS.

Utiliser Dictionary

Lorsque : on associe une clé à une valeur, on veut retrouver une valeur grâce à une clé.

Exemples : Paramètres, Informations de profil, Compteurs par catégorie, Configuration.

Utiliser Set

Lorsque : les valeurs doivent être uniques, on veut tester rapidement l'appartenance.

Exemples : Permissions, Identifiants uniques, Catégories sélectionnées, Fonctionnalités activées.

52. Les tuples

Un tuple permet de regrouper plusieurs valeurs. Contrairement à un tableau, ces valeurs peuvent être de types différents.

Exemple :

```
let utilisateur = (
    "Alex",
    25,
```

```
    true
)
```

Le tuple contient :

```
String
Int
Bool
```

53. Accéder aux valeurs d'un tuple

```
let utilisateur = (
    "Alex",
    25,
    true
)

print(utilisateur.0)
print(utilisateur.1)
print(utilisateur.2)
```

Résultat :

```
Console
Alex
25
true
```

Cette notation fonctionne, mais elle est peu descriptive.

54. Tuple nommé

Il est préférable de nommer les éléments.

```
let utilisateur = (
    nom: "Alex",
    age: 25,
    estConnecte: true
)
```

Utilisation :

```
print(utilisateur.nom)
print(utilisateur.age)
print(utilisateur.estConnecte)
```

Le code devient beaucoup plus lisible.

55. Exemple — État d'un appareil

```
let appareil = (
    modele: "iPhone",
    batterie: 82,
    wifi: true
)
```

Puis :

```
print("Appareil : \(appareil.modele)")
print("Batterie : \(appareil.batterie)%")
print("Wi-Fi : \(appareil.wifi)")
```

56. Décomposer un tuple

```
let position = (
    latitude: 45.5019,
    longitude: -73.5674
)
```

On peut écrire :

```
let (latitude, longitude) = position
```

Puis :

```
print(latitude)
print(longitude)
```

57. Ignorer une valeur

```
let utilisateur = (
    nom: "Alex",
    age: 25,
    ville: "Montréal"
)
```

Si l'âge ne nous intéresse pas :

```
let (nom, _, ville) = utilisateur
```

Le symbole _ permet d'ignorer une valeur.

58. Tuple ou collection ?

Un tuple est généralement adapté lorsqu'on regroupe quelques valeurs ayant des rôles différents.

Exemple :

```
let position = (
    latitude: 45.5,
    longitude: -73.5
)
```

Un tableau est préférable pour une quantité variable d'éléments du même type.

Exemple :

```
let positions: [Double] = [
    45.5,
    46.2,
    47.1
]
```

59. Combiner les collections

Les collections peuvent être combinées.

Exemple :

```
var utilisateurs: [[String: String]] = [
    [
        "nom": "Alex",
        "ville": "Montréal"
    ],
    [
        "nom": "Marie",
        "ville": "Laval"
    ]
]
```

Ici :

```
Array
  ↓
contient plusieurs
  ↓
Dictionary
```

Ce type de structure sera fréquent dans des applications réelles.

60. Tableau de tuples

On peut également écrire :

```
let appareils = [
    (
        nom: "iPhone",
        batterie: 80
    ),
    (
        nom: "iPad",
        batterie: 65
    ),
    (
        nom: "Apple Watch",
        batterie: 40
    )
]
```

Puis :

```
for appareil in appareils {
    print(
        "\(appareil.nom) : \(appareil.batterie)%"
    )
}
```

61. Exemple — Liste d'appareils

```
let appareils = [
    (
        nom: "iPhone",
        batterie: 90,
        connecte: true
    ),
    (
        nom: "iPad",
        batterie: 50,
        connecte: false
    ),
    (
        nom: "Apple Watch",
        batterie: 25,
        connecte: true
    )
]
```

Parcours :

```swift
for appareil in appareils {
    print("Appareil : \(appareil.nom)")
    print("Batterie : \(appareil.batterie)%")
    if appareil.connecte {
        print("État : connecté")
    } else {
        print("État : déconnecté")
    }
    print()
}
```

62. Exemple guidé — Centre de notifications

Créer :

```swift
var notifications: [String] = [
    "Nouveau message",
    "Mise à jour disponible",
    "Batterie faible"
]
```

Ajouter :

```swift
"Synchronisation terminée"
```

Afficher le nombre de notifications. Afficher ensuite toutes les notifications avec leur numéro :

```
Console
1. Nouveau message
2. Mise à jour disponible
3. Batterie faible
4. Synchronisation terminée
```

Puis supprimer une notification et afficher la nouvelle liste.

63. Exemple guidé — Paramètres de l'application

Créer :

```swift
var parametres: [String: String] = [
    "theme": "sombre",
    "langue": "fr",
    "unite": "km"
]
```

Afficher :

```
Console
Thème : sombre
Langue : fr
```

```
Unité : km
```

Modifier ensuite theme pour clair. Ajouter :

```
notifications → activees
```

64. Exercice — Gestion d'une application

On souhaite représenter plusieurs informations d'une application.

1. Notifications

Créer un tableau :

```
var notifications: [String]
```

avec au moins quatre notifications. Afficher :

```
Console
Nombre de notifications : ...
```

Puis afficher toutes les notifications avec une boucle.

2. Notifications prioritaires

Pendant le parcours du tableau, si une notification contient le mot "Urgent", afficher :

```
Console
[IMPORTANT] ...
```

Sinon, afficher normalement la notification.

Indice :

```
contains()
```

3. Paramètres

Créer un dictionnaire contenant : theme, langue, uniteDistance.

Exemple :

```
theme         → sombre
langue        → fr
uniteDistance → km
```

Afficher chaque paramètre.

4. Valeur absente

Essayer de récupérer volume dans le dictionnaire. Si cette clé n'existe pas, utiliser 50 comme valeur par défaut.

Utiliser :

```
??
```

5. Permissions

Créer :

```
let permissionsRequises: Set<String>
```

contenant : camera, location, notifications.

Puis :

```
let permissionsAccordees: Set<String>
```

contenant seulement : camera, notifications.

Déterminer et afficher les permissions manquantes.

6. Informations sur l'appareil

Créer un tuple :

```
let appareil = (
    modele: ...,
    batterie: ...,
    connecte: ...
)
```

Afficher un résumé :

```
Console
------ APPAREIL ------
Modèle : iPhone
Batterie : 75 %
Connecté : true
```

65. Exercice — Historique de batterie

Créer :

```
let historiqueBatterie = [
    100,
    90,
    75,
    60,
    45,
    20,
    10
]
```

Parcourir le tableau. Pour chaque valeur :

Niveau	État
Supérieure à 50	Normal

21 à 50	Faible
20 ou moins	Critique

Afficher par exemple :

```
Console
100 % → Normal
90 % → Normal
45 % → Faible
20 % → Critique
```

66. Exercice — Recherche dans une collection

Créer :

```
let appareils = [
    "iPhone",
    "iPad",
    "Apple Watch",
    "Mac"
]
```

Créer ensuite :

```
let recherche = "iPad"
```

Vérifier si l'appareil existe.

Si oui :

```
Console
iPad trouvé
```

Sinon :

```
Console
Appareil introuvable
```

67. À retenir

Array

```
var valeurs: [String] = []
```

Caractéristiques : ordonné, indexé, doublons autorisés.

Dictionary

```
var valeurs: [String: Int] = [:]
```

Caractéristiques : clé → valeur, recherche par clé, accès retournant souvent un optional.

Set

```
var valeurs: Set<String> = []
```

Caractéristiques : valeurs uniques, pas d'ordre logique à supposer, recherche d'appartenance efficace.

Tuple

```
let valeur = (
    nom: "Alex",
    age: 25
)
```

Caractéristiques : regroupe plusieurs valeurs, types différents possibles, nombre d'éléments fixe, éléments nommables.

68. Vision globale

Les notions étudiées commencent maintenant à se combiner :

```
Variables
    ↓
Conditions
    ↓
Boucles
    ↓
Fonctions
    ↓
Optionnels
    ↓
Collections
```

On peut désormais représenter et manipuler :

- Une valeur
- Plusieurs valeurs
- Des valeurs éventuellement absentes
- Des associations clé / valeur
- Des ensembles uniques
- Des groupes de données simples